

# Windows Service & Process Monitoring Agent

Defensive Endpoint Monitoring | Process, Service, and Alert Detection

## Cybersecurity Internship Project

|                    |                          |                              |                                     |
|--------------------|--------------------------|------------------------------|-------------------------------------|
| 267                | 293                      | 149                          | 0                                   |
| Processes observed | Windows services checked | Medium alerts in current run | High/Critical alerts in current run |

### Analyst Note

The 149 medium alerts shown in the captured run are primarily baseline/allowlist findings. They indicate processes requiring review, not confirmed malicious activity.

Prepared for internship project submission

# 1. Executive Summary

This project implements a lightweight endpoint monitoring and detection workflow for Windows systems. It collects process and service telemetry, evaluates parent-child relationships, identifies processes outside a configurable baseline, checks executable locations, and writes structured reports for analyst review.

The implementation is intentionally rule-based and transparent. Each alert carries the evidence available to the detector - process name, PID/PPID, path, service information, timestamp, and reason - so that an analyst can validate the finding instead of treating the alert as an automatic malware verdict.

## SOC Perspective

The project mirrors a simplified endpoint detection workflow: collect telemetry -> normalize -> apply detection rules -> score findings -> preserve evidence -> visualize in a monitoring console.

# 2. Problem Statement

Windows endpoints expose several security-relevant artifacts during normal operation. Malware, persistence mechanisms, and unauthorized software can abuse processes, services, startup configuration, and user-writable locations. A small defensive monitor can provide useful visibility by continuously checking these artifacts and surfacing deviations from an expected baseline.

# 3. Objectives

1. Enumerate active Windows processes and collect useful endpoint metadata.
2. Build parent-child process relationships using PID and PPID.
3. Detect process relationships that warrant investigation.
4. Audit Windows services, startup mode, accounts, and executable paths.
5. Detect processes not present in the configured allowlist.
6. Detect executables launched from suspicious user-writable paths.
7. Generate repeatable CSV and JSON evidence for analyst review.
8. Provide a Streamlit dashboard for SOC-style visibility.

# 4. Scope and Use Case

The current implementation is designed for authorized defensive testing on a Windows workstation. It is a learning-oriented monitoring agent rather than a production EDR platform. The main use case is endpoint triage: identify unusual process/service activity, preserve context, and provide an analyst-friendly view of findings.

## 5. SOC Monitoring Architecture

| Layer        | Component                      | Security Role                                                     |
|--------------|--------------------------------|-------------------------------------------------------------------|
| Collection   | Process Monitor                | Collect PID, PPID, name, path, user, status, timestamp            |
| Collection   | Windows Service Monitor        | Enumerate services, state, startup mode, account, executable path |
| Detection    | Parent-Child Analyzer          | Match process lineage against suspicious relationship rules       |
| Detection    | Unauthorized Process Detector  | Compare names against allowlist and inspect executable paths      |
| Decision     | Severity / Alert Engine        | Normalize detection findings into analyst-readable alerts         |
| Presentation | CSV/JSON + Streamlit Dashboard | Persist evidence and provide SOC-style visibility                 |

### Data Flow

Windows endpoint -> collection modules -> rule-based detections -> severity classification -> CSV/JSON evidence -> Streamlit dashboard -> analyst review.

## 6. Detection Methodology

### 6.1 Process Telemetry

The process monitor uses psutil to iterate over running processes. It attempts to collect PID, PPID, process name, executable path, username, status, and collection time. Access-denied and terminated-process conditions are handled so that one protected process does not stop the full scan.

### 6.2 Parent-Child Detection

The process tree is reconstructed from PID/PPID relationships. A configurable rule set checks combinations such as Microsoft Office applications launching command interpreters or scripting engines. In the captured host scan no suspicious relationship was present, while direct rule testing confirmed the detector logic behaves as intended.

### 6.3 Service Auditing

Windows services are queried through PowerShell/Windows Management Instrumentation. The monitor captures service name, display name, state, startup mode, start account, and executable path. Paths containing common user-writable locations are treated as potentially suspicious for further review.

### 6.4 Unauthorized Process Detection

The allowlist module defines common expected process names. A process outside the baseline is assigned a medium-level investigation finding by default; suspicious executable locations can increase the risk level. The current screenshots show a high number of medium findings because the baseline is intentionally small and requires tuning for the host.

### 6.5 Continuous Monitoring

The continuous monitor repeats the collection and detection cycle every ten seconds. A lightweight change detector compares PIDs between scans and highlights newly observed processes.

## 7. Implementation Components

| Module                  | Purpose                                                                      |
|-------------------------|------------------------------------------------------------------------------|
| process_monitor.py      | Enumerates active processes and collects endpoint metadata.                  |
| process_tree.py         | Builds process relationships and evaluates parent-child rules.               |
| detection_rules.py      | Stores suspicious parent-child relationship rules.                           |
| service_monitor.py      | Collects Windows service inventory through PowerShell/WMI.                   |
| service_rules.py        | Identifies suspicious service executable paths.                              |
| unauthorized_process.py | Applies process allowlist and suspicious-path detections.                    |
| severity.py             | Provides a reusable severity scoring model.                                  |
| alert_manager.py        | Combines detection results into a consolidated JSON alert file.              |
| monitor.py              | Runs one complete endpoint monitoring cycle and updates reports.             |
| continuous_monitor.py   | Runs repeated monitoring cycles and detects process changes.                 |
| dashboard/app.py        | Presents metrics, filtered alerts, process inventory, and service inventory. |

## 8. Reporting and Evidence

- process\_report.csv - current process inventory and telemetry.
- service\_report.csv - current Windows service inventory.
- alerts.json - consolidated alert evidence for dashboard consumption.
- monitoring\_summary.json - current scan counts and severity totals.
- screenshots/ - visual evidence used in the internship report and GitHub repository.

## 9. Test Results from Captured Run

| Test Area                      | Observed Result                                                   | SOC Interpretation                                                                           |
|--------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Process enumeration            | 267 processes detected                                            | Endpoint process visibility is working.                                                      |
| Service audit                  | 293 services checked                                              | Service inventory collection is working.                                                     |
| Parent-child runtime scan      | 0 alerts                                                          | No configured suspicious relationship was observed during the captured scan.                 |
| Service path detection         | 0 alerts                                                          | No configured suspicious service path was observed during the captured scan.                 |
| Unauthorized process detection | 149 medium findings                                               | Baseline/allowlist is currently under-tuned and should be refined to reduce false positives. |
| Continuous monitoring          | 3 newly observed processes in captured cycle                      | Change detection is identifying process starts between scans.                                |
| Dashboard                      | Metrics, charts, alerts, process and service inventories rendered | Evidence is consumable through a SOC-style analyst interface.                                |

### Key Finding

The strongest current improvement area is baseline tuning. The detector is operational, but a production-quality allowlist should be built from the normal software inventory of the specific endpoint or organization.

## 10. Implementation Evidence - Screenshots

The following figures document the implemented modules and the captured SOC-style dashboard. All seven screenshots supplied with the project are included below.

## Figure 1 - Process Monitoring Output

```
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> python agent\process_monitor.py

=====
WINDOWS PROCESS MONITOR
=====
PID      PPID     PROCESS NAME          STATUS
-----
0         0        System Idle Process    running
4         0        System                 running
236       4        Unknown                stopped
280       4        Registry               running
932       1504     svchost.exe            running
944       4        smss.exe               running
980       1484     dwm.exe                running
1296      1084     csrss.exe              running
1384      1084     wininit.exe            running
1392      1376     csrss.exe              running
1484      1376     winlogon.exe           running
1504      1384     services.exe           running
1536      1384     LsaIso.exe             running
1556      1384     lsass.exe              running
1616      1504     svchost.exe            running
1628      1504     WUDFHost.exe           running
1660      1700     SystemSettings.exe     stopped
1700      1504     svchost.exe            running
1732      1384     fontdrvhost.exe        running
1740      1484     fontdrvhost.exe        running
1748      1504     WUDFHost.exe           running
1816      1504     elevation_service.exe   running
1856      1504     svchost.exe            running
1916      1504     svchost.exe            running
1924      1504     svchost.exe            running
1960      1504     WUDFHost.exe           running
1996      9540     msedge.exe             running
2020      1504     svchost.exe            running
```

*The agent enumerates active Windows processes and captures PID, PPID, process name, and runtime status.*

## Figure 2 - Parent-Child Detection Validation

```
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> python agent\process_tree.py

=====
PROCESS RELATIONSHIP ALERTS
=====
No suspicious parent-child relationships detected.

Total alerts: 0
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> python -c "from agent.detection_rules import is_suspicious_paren
t_child; print('winword.exe -> powershell.exe:', is_suspicious_parent_child('winword.exe','powershell.exe')); print('explorer.exe -> notepa
d.exe:', is_suspicious_parent_child('explorer.exe','notepad.exe'))"
winword.exe -> powershell.exe: True
explorer.exe -> notepad.exe: False
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor>
```

*The runtime scan returned no suspicious relationship on the host, while the rule validation confirms that winword.exe -> powershell.exe is treated as suspicious and explorer.exe -> notepad.exe is not.*

### Figure 3 - Windows Service Audit

```
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> python agent\service_monitor.py

=====
WINDOWS SERVICE AUDIT
=====
SERVICE NAME      STATE      START MODE  START ACCOUNT
-----
ADPSvc             Stopped    Manual      NT AUTHORITY\LocalServi
ALG                Stopped    Manual      NT AUTHORITY\LocalServi
AppHostSvc         Running    Auto        localSystem
AppIDSvc           Running    Manual      NT Authority\LocalServi
Appinfo            Running    Manual      LocalSystem
AppReadiness       Stopped    Manual      LocalSystem
AppXSvc            Stopped    Auto        LocalSystem
ApxSvc             Stopped    Manual      NT AUTHORITY\LocalServi
aspnet_state       Stopped    Manual      NT AUTHORITY\NetworkSer
AudioEndpointBuilder Running    Auto        LocalSystem
Audiosrv           Running    Auto        NT AUTHORITY\LocalServi
autotimesvc        Stopped    Manual      NT AUTHORITY\LocalServi
AxInstSV           Stopped    Manual      LocalSystem
BDESVC             Stopped    Manual      localSystem
BFE                Running    Auto        NT AUTHORITY\LocalServi
BITS               Stopped    Manual      LocalSystem
brave              Stopped    Auto        LocalSystem
BraveElevationService Stopped    Manual      LocalSystem
bravem             Stopped    Manual      LocalSystem
BrokerInfrastructure Running    Auto        LocalSystem
BTAGService        Stopped    Manual      NT AUTHORITY\LocalServi
```

The service module enumerates Windows services and records state, startup mode, and service account information.



## Figure 4 - Unauthorized Process Detection

```
PID      : 20972
PPID     : 17608
Path     : C:\Program Files (x86)\Dropbox\Client\Dropbox.exe
User     : VASANTHA\nitin
Reason   : Process is not present in the configured allowlist.

[MEDIUM]
Process  : Dropbox.exe
PID      : 21016
PPID     : 17608
Path     : C:\Program Files (x86)\Dropbox\Client\Dropbox.exe
User     : VASANTHA\nitin
Reason   : Process is not present in the configured allowlist.

[MEDIUM]
Process  : Code.exe
PID      : 21080
PPID     : 5912
Path     : C:\Users\nitin\AppData\Local\Programs\Microsoft VS Code\Code.exe
User     : VASANTHA\nitin
Reason   : Process is not present in the configured allowlist.

[MEDIUM]
Process  : SearchFilterHost.exe
PID      : 21180
PPID     : 12352
Path     : C:\Windows\System32\SearchFilterHost.exe
User     : None
Reason   : Process is not present in the configured allowlist.

Total alerts: 147
❖ (venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> |
```

*The allowlist-based detector flags processes not currently present in the configured baseline. The result is an investigation queue, not an automatic malware verdict.*

## Figure 5 - Continuous Monitoring

```
Scan time: 2026-08-17 17:58:47

[1/5] Collecting processes...
      Processes detected: 267

[2/5] Detecting process changes...
      New processes detected: 3
      NEW PROCESS: SearchFilterHost.exe (PID 8052)
      NEW PROCESS: SearchProtocolHost.exe (PID 10572)
      NEW PROCESS: SearchFilterHost.exe (PID 20132)

[3/5] Analyzing parent-child relationships...
      Parent-child alerts: 0

[4/5] Auditing Windows services...
      Services checked: 293
      Service alerts: 0

[5/5] Detecting unauthorized/suspicious processes...
      Unauthorized/suspicious process alerts: 151

-----
SCAN SUMMARY
-----
Processes detected      : 267
New processes          : 3
Services checked       : 293
Parent-child alerts    : 0
Service alerts         : 0
Unauthorized alerts    : 151
Total security alerts  : 151
Alert report           : reports\alerts.json
-----

Next scan in 10 seconds...
█
```

*The continuous monitor performs repeated collection cycles and detects newly observed processes between scans.*

## Figure 6 - Security Alert Report

```
(venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> python agent\monitor.py

=====
WINDOWS SERVICE & PROCESS MONITORING AGENT
=====

[1/4] Collecting Windows processes...
      267 processes collected.
      Process report saved to: reports\process_report.csv

[2/4] Analyzing parent-child relationships...
      0 parent-child alerts detected.

[3/4] Auditing Windows services...
      293 services checked.
      Service report saved to: reports\service_report.csv
      0 service alerts detected.

[4/4] Detecting unauthorized/suspicious processes...
      149 unauthorized/suspicious process alerts detected.

=====
MONITORING SUMMARY
=====
Timestamp           : 2026-08-17 18:00:11
Processes detected   : 267
Services detected    : 293
Process relationship alerts : 0
Service alerts       : 0
Unauthorized process alerts : 149
Total alerts         : 149
Critical alerts      : 0
High severity alerts : 0
Medium severity alerts : 149
Low severity alerts  : 0

=====
MONITORING COMPLETE
=====
Process report : reports\process_report.csv
Service report : reports\service_report.csv
Summary report : reports\monitoring_summary.json
Alert report   : reports\alerts.json
❖ (venv) PS C:\Users\nitin\OneDrive\Desktop\Windows-Service-Process-Monitor> |
```

*The generated alert report provides structured evidence for downstream review and dashboard visualization.*

Figure 7 - SOC-Style Monitoring Dashboard



The Streamlit dashboard summarizes process and service counts, alert totals, severity distribution, alert type distribution, and endpoint inventory.

11. SOC Analyst Interpretation

From a SOC workflow perspective, the project demonstrates the basic pattern of endpoint triage. The analyst receives a normalized finding, reviews the process or service context, checks lineage and executable location, and then decides whether the event is benign, suspicious, or worthy of deeper investigation.

- A parent-child alert should be validated against the expected application workflow before escalation.
- A service executing from a user-writable path should be reviewed for installation origin, signer, owner, and persistence intent.
- An unauthorized-process finding should be correlated with the software inventory before it is treated as suspicious.
- Medium alerts are useful investigation leads, but excessive volume indicates the endpoint baseline needs tuning.

12. Limitations

- The current allowlist is intentionally small and therefore produces false positives on a normal developer/workstation environment.
- The project does not yet verify digital signatures or calculate file hashes.
- The project does not ingest Windows Event Logs, network telemetry, or threat intelligence.
- The dashboard is a local Streamlit interface and is not a complete enterprise EDR console.

- Alerts are investigation triggers and not definitive malware classifications.

## 13. Future Enhancements

- Build the allowlist dynamically from an approved software baseline.
- Add Authenticode signature verification and SHA-256 hashes.
- Integrate Windows Event Logs and PowerShell logs.
- Add network connection telemetry and threat-intelligence enrichment.
- Persist alerts in a database for historical search and trend analysis.
- Add authentication, role-based access, and alert acknowledgement to the dashboard.
- Add notification channels such as email or webhook-based alerting.

## 14. Conclusion

The Windows Service & Process Monitoring Agent successfully demonstrates a practical defensive monitoring workflow for Windows endpoints. It collects process and service telemetry, evaluates transparent detection rules, generates structured evidence, and presents the results in a dashboard suitable for a small SOC-style demonstration. The project also highlights an important security engineering principle: detection quality depends not only on rules, but on a well-maintained baseline and analyst validation.

## 15. Project Deliverables

- GitHub repository containing the monitoring agent and dashboard source code.
- Detailed project report in PDF and editable DOCX formats.
- Seven implementation screenshots for evidence and documentation.
- Streamlit dashboard intended for deployment as the project demonstration URL.
- Feedback video link to be recorded and uploaded for final internship submission.

**END OF REPORT**